



Azkaban 工作流调度



一、 课程计划

目录

一、 课程计划.....	2
二、 工作流.....	4
1. 工作流产生背景.....	4
2. 工作流调度实现方式.....	4
3. 工作流调度工具之间对比.....	5
三、 Azkaban 调度器.....	6
1. Azkaban 介绍.....	6
2. Azkaban 原理架构.....	7
3. Azkaban 三种部署模式.....	7
3.1. solo server mode.....	7
3.2. two-server mode.....	7
3.3. multiple-executor mode.....	7
4. Azkaban 源码编译.....	8
4.1. 编译环境.....	8
4.2. 下载源码解压.....	8
4.3. 编译源码.....	8
4.4. 编译后安装包路径.....	9
四、 Azkaban 安装部署.....	10
1. solo-server 模式部署.....	10
1.1. 节点规划.....	10
1.2. 解压配置.....	10
1.3. 启动验证.....	11
1.4. 登录 web 页面.....	11
1.5. 初体验测试.....	12
2. two-server 模式部署.....	14
2.1. 节点规划.....	14
2.2. mysql 配置初始化.....	14
2.3. web-server 服务器配置.....	15
2.4. exec-server 服务器配置.....	17
2.5. 集群启动.....	18
3. multiple-executor 模式部署.....	19
3.1. 节点规划.....	19
3.2. scp executor server 安装包到 node-3.....	19
五、 Azkaban 使用实战.....	20



1. shell command 调度	20
2. job 依赖调度	21
3. HDFS 任务调度	22
4. MAPREDUCE 任务调度	22
5. HIVE 脚本任务调度	23
6. 定时任务调度	24



二、 workflow

1. workflow产生背景

workflow (Workflow)，指“业务过程的部分或整体在计算机应用环境下的自动化”。是对 workflow 及其各操作步骤之间业务规则的抽象、概括描述。workflow 解决的主要问题是：为了实现某个业务目标，利用计算机软件在多个参与者之间按某种预定规则自动传递文档、信息或者任务。

一个完整的数据分析系统通常都是由多个前后依赖的模块组合构成的：数据采集、数据预处理、数据分析、数据展示等。各个模块单元之间存在时间先后依赖关系，且存在着周期性重复。

为了很好地组织起这样的复杂执行计划，需要一个 workflow 调度系统来调度执行。

2. workflow调度实现方式

简单的任务调度：直接使用 linux 的 crontab 来定义,但是缺点也是比较明显，无法设置依赖。

复杂的任务调度：自主开发调度平台，使用开源调度系统，比如 azkaban、Apache Oozie、Cascading、Hamake 等。

其中知名度比较高的是 Apache Oozie，但是其配置 workflow 的过程是编写大量的 XML 配置，而且代码复杂度比较高，不易于二次开发。

3. workflow调度工具之间对比

下面的表格对四种 hadoop workflow调度器的关键特性进行了比较，尽管这些workflow调度器能够解决的需求场景基本一致，但在设计理念，目标用户，应用场景等方面还是存在显著的区别，在做技术选型的时候，可以提供参考。

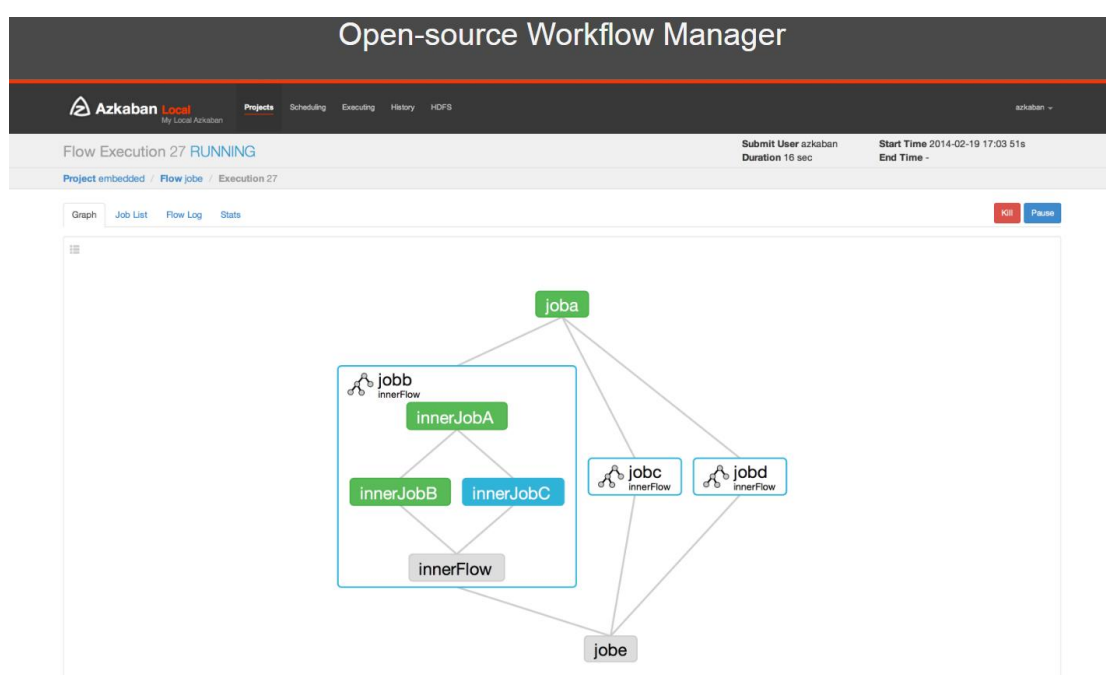
特性	Hamake	Oozie	Azkaban	Cascading
workflow描述语言	XML	XML (xPDL based)	text file with key/value pairs	Java API
依赖机制	data-driven	explicit	explicit	explicit
是否要 web 容器	No	Yes	Yes	No
进度跟踪	console/log messages	web page	web page	Java API
Hadoop job 调度支持	no	yes	yes	yes
运行模式	command line utility	daemon	daemon	API
Pig 支持	yes	yes	yes	yes
事件通知	no	no	no	yes
需要安装	no	yes	yes	no
支持的 hadoop 版本	0.18+	0.20+	currently unknown	0.18+
重试支持	no	workflownode evel	yes	yes
运行任意命令	yes	yes	yes	yes
Amazon EMR 支持	yes	no	currently unknown	yes



三、 Azkaban 调度器

1. Azkaban 介绍

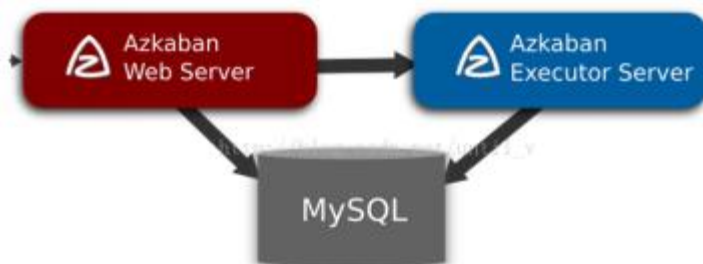
Azkaban 是由 linkedin（领英）公司推出的一个批量 workflow 任务调度器，用于在一个 workflow 内以一个特定的顺序运行一组工作和流程。Azkaban 使用 job 配置文件建立任务之间的依赖关系，并提供一个易于使用的 web 用户界面维护和跟踪你的 workflow。



Azkaban 功能特点：

- 提供功能清晰，简单易用的 Web UI 界面
- 提供 job 配置文件快速建立任务和任务之间的依赖关系
- 提供模块化和可插拔的插件机制，原生支持 command、Java、Hive、Pig、Hadoop
- 基于 Java 开发，代码结构清晰，易于二次开发

2. Azkaban 原理架构



mysql 服务器: 存储元数据，如项目名称、项目描述、项目权限、任务状态、SLA 规则等

AzkabanWebServer: 对外提供 web 服务，使用户可以通过 web 页面管理。职责包括项目管理、权限授权、任务调度、监控 executor。

AzkabanExecutorServer: 负责具体的工作流的提交、执行。

3. Azkaban 三种部署模式

3.1. solo server mode

该模式中 webServer 和 executorServer 运行在**同一个进程中**，进程名是 AzkabanSingleServer。使用**自带的 H2 数据库**。这种模式包含 Azkaban 的所有特性，但一般用来**学习和测试**。

3.2. two-server mode

该模式使用 MySQL 数据库，Web Server 和 Executor Server 运行在**不同的进程**中。

3.3. multiple-executor mode

该模式使用 MySQL 数据库，Web Server 和 Executor Server 运行在**不同的机器**中。且**有多个 Executor Server**。该模式适用于大规模应用。



4. Azkaban 源码编译

Azkaban3.x 在安装前需要自己编译成二进制包。

并且提前安装好 Maven、Ant、Node 等软件，具体请参考附件资料

4.1. 编译环境

```
yum install -y git
```

```
yum install -y gcc-c++
```

4.2. 下载源码解压

```
wget https://github.com/azkaban/azkaban/archive/3.51.0.tar.gz
```

```
tar -zxvf 3.51.0.tar.gz
```

```
cd ./azkaban-3.51.0/
```

4.3. 编译源码

```
./gradlew build installDist -x test
```

Gradle 是一个基于 Apache Ant 和 Apache Maven 的项目自动化构建工具。

-x test 跳过测试。（注意联网下载 jar 可能会失败、慢）

```
> Task :azkaban-web-server:npmSetup
/root/build/azkaban-3.51.0/azkaban-web-server/.gradle/npm/npm-v5.6.0/b-
ver/.gradle/npm/npm-v5.6.0/lib/node_modules/npm/bin/npm-cli.js
/root/build/azkaban-3.51.0/azkaban-web-server/.gradle/npm/npm-v5.6.0/b-
ver/.gradle/npm/npm-v5.6.0/lib/node_modules/npm/bin/npm-cli.js
+ npm@5.6.0
added 476 packages in 10.527s

> Task :azkaban-web-server:npm_install
added 39 packages in 1.282s

BUILD SUCCESSFUL in 38s
77 actionable tasks: 63 executed, 14 from cache
[root@node-1 azkaban-3.51.0]#
```




4.4. 编译后安装包路径

编译成功之后就可以在指定的路径下取得对应的安装包了。

```
[root@node-1 azkaban-3.51.0]# pwd
/root/build/azkaban-3.51.0
[root@node-1 azkaban-3.51.0]# ll azkaban-solo-server/build/distributions/
total 46756
-rw-r--r--. 1 root root 23877219 May 29 15:47 azkaban-solo-server-0.1.0-SNAPSHOT.tar.gz
-rw-r--r--. 1 root root 23998107 May 29 15:47 azkaban-solo-server-0.1.0-SNAPSHOT.zip
[root@node-1 azkaban-3.51.0]# ll azkaban-web-server/build/distributions/
total 39196
-rw-r--r--. 1 root root 20011432 May 29 15:47 azkaban-web-server-0.1.0-SNAPSHOT.tar.gz
-rw-r--r--. 1 root root 20122812 May 29 15:47 azkaban-web-server-0.1.0-SNAPSHOT.zip
[root@node-1 azkaban-3.51.0]# ll azkaban-exec-server/build/distributions/
total 30796
-rw-r--r--. 1 root root 15762309 May 29 15:47 azkaban-exec-server-0.1.0-SNAPSHOT.tar.gz
-rw-r--r--. 1 root root 15767358 May 29 15:47 azkaban-exec-server-0.1.0-SNAPSHOT.zip
[root@node-1 azkaban-3.51.0]#
```

#solo-server 模式安装包路径

azkaban-solo-server/build/distributions/

#two-server 模式和 multiple-executor 模式 web-server 安装包路径

azkaban-web-server/build/distributions/

#two-server 模式和 multiple-executor 模式 exec-server 安装包路径

azkaban-exec-server/build/distributions/

```
[root@node-1 azkaban-3.51.0]# ll azkaban-db/build/distributions/
total 16
-rw-r--r--. 1 root root 4069 May 29 15:47 azkaban-db-0.1.0-SNAPSHOT.tar.gz
-rw-r--r--. 1 root root 11786 May 29 15:47 azkaban-db-0.1.0-SNAPSHOT.zip
[root@node-1 azkaban-3.51.0]#
```

azkaban-db/build/distributions/

数据库相关安装包路径



四、 Azkaban 安装部署

1. solo-server 模式部署

1.1. 节点规划

HOST	角色
node-1	Web Server 和 Executor Server 同一进程

1.2. 解压配置

```
mkdir /export/servers/azkaban
```

```
tar -zxvf azkaban-solo-server-0.1.0-SNAPSHOT.tar.gz -C /export/servers/azkaban/
```

```
vim conf/azkaban.properties
```

```
default.timezone.id=Asia/Shanghai #修改时区
```

```
vim plugins/jobtypes/commonprivate.properties
```

```
添加：memCheck.enabled=false
```

```
azkaban 默认需要 3G 的内存，剩余内存不足则会报异常
```

1.3. 启动验证

```
cd azkaban-solo-server-0.1.0-SNAPSHOT/
```

```
bin/start-solo.sh
```

注:启动/关闭必须进到 azkaban-solo-server-0.1.0-SNAPSHOT/目录下。

```
[root@node-1 azkaban-solo-server-0.1.0-SNAPSHOT]# bin/start-solo.sh
[root@node-1 azkaban-solo-server-0.1.0-SNAPSHOT]# jps
26944 GradleDaemon
27990 AzkabanSingleServer
4375 ResourceManager
4503 NodeManager
4104 DataNode
3961 NameNode
28027 Jps
```

AzkabanSingleServer(对于 Azkaban solo-server 模式, Exec Server 和 Web Server 在同一个进程中)

1.4. 登录 web 页面

访问 Web Server=><http://node-1:8081/> 默认用户名密码 **azkaban**

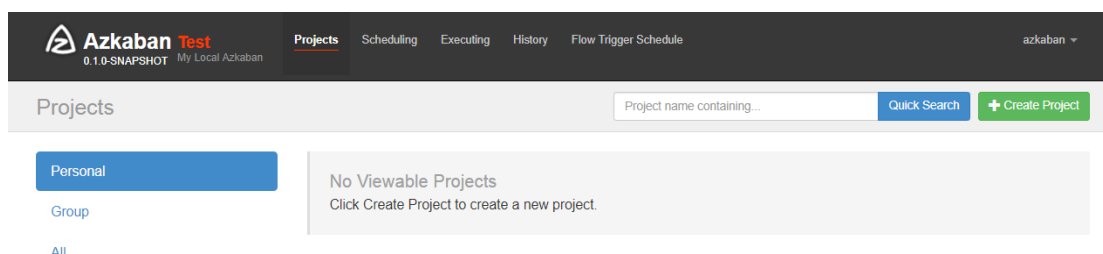


Login

azkaban	默认用户名
.....	密码

azkaban

Login



1.5. 初体验测试

http://node-1:8081/index 登录=>Create Project=>Upload zip 包 =>execute flow
执行一步步操作即可。

创建两个文件 one.job two.job,内容如下，打包成 zip 包。

cat one.job

type=command

command=echo "this is job one"

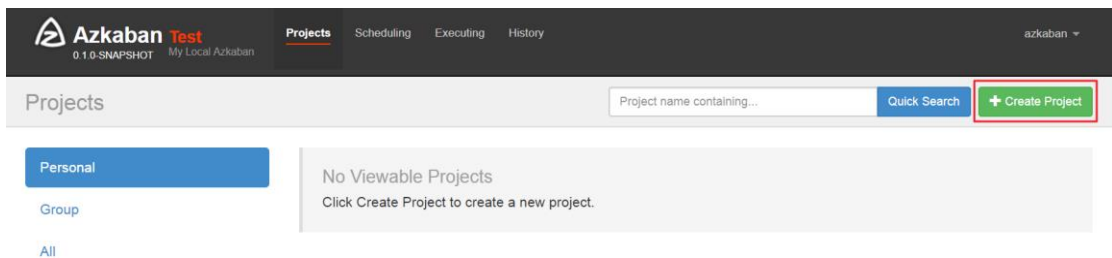
cat two.job

type=command

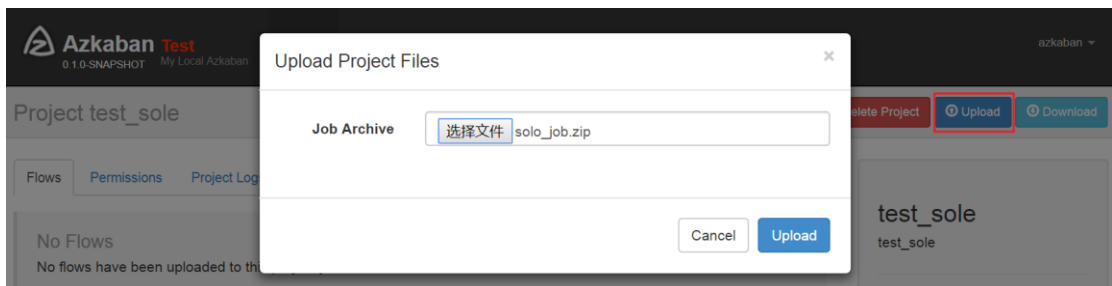
dependencies=one

command=echo "this is job two"

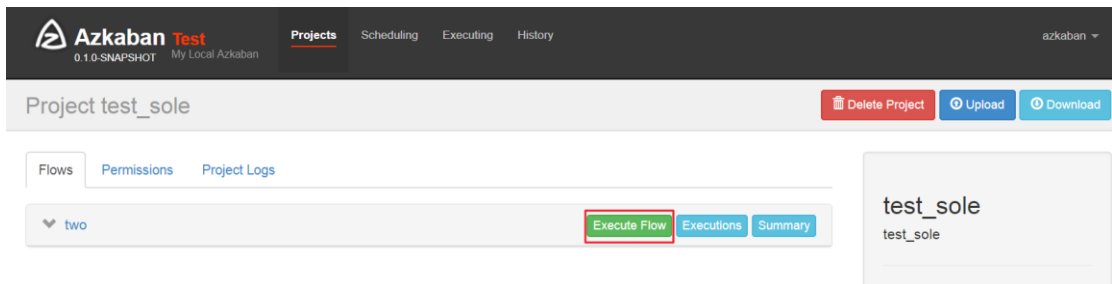
创建工程：



上传 zip 压缩包：



execute 执行：



执行页面：

Flow View

Right click on the jobs to disable and enable jobs in the flow.

[Notification](#)

[Failure Options](#)

[Concurrent](#)

[Flow Parameters](#)



[Schedule](#) [Cancel](#) [Execute](#)

执行结果查看：

[Graph](#) [Flow Trigger List](#) [Job List](#) [Flow Log](#) [Stats](#) [Prepare Execution](#)

Name	Type	Timeline	Start Time	End Time	Elapsed	Status	Details
one	command		2019-05-29 16:42:00s	2019-05-29 16:42:00s	0 sec	Success	Details
two	command		2019-05-29 16:42:00s	2019-05-29 16:42:00s	0 sec	Success	Details

[Job Logs](#)

Job Logs

```
29-05-2019 16:42:00 CST one INFO - Starting job one at 1559119320435
29-05-2019 16:42:00 CST one INFO - azkaban.webserver.url property was not set
29-05-2019 16:42:00 CST one INFO - job JVM args: -Dazkaban.flowid=two -Dazkaban.execid=3 -Dazkaban.jobid=one
29-05-2019 16:42:00 CST one INFO - user.to.proxy property was not set, defaulting to submit user azkaban
29-05-2019 16:42:00 CST one INFO - Building command job executor.
29-05-2019 16:42:00 CST one INFO - 1 commands to execute.
29-05-2019 16:42:00 CST one INFO - cwd=/export/servers/azkaban/azkaban-solo-server-0.1.0-SNAPSHOT/executions/3
29-05-2019 16:42:00 CST one INFO - effective user is: azkaban
29-05-2019 16:42:00 CST one INFO - Command: echo "this is job one"
29-05-2019 16:42:00 CST one INFO - Environment variables: {JOB_OUTPUT_PROP_FILE=/export/servers/azkaban/azkaban-solo-server-0.1.0-SNAPSHOT/executions/3/one_output_37
29-05-2019 16:42:00 CST one INFO - Working directory: /export/servers/azkaban/azkaban-solo-server-0.1.0-SNAPSHOT/executions/3
29-05-2019 16:42:00 CST one INFO - this is job one
29-05-2019 16:42:00 CST one INFO - Process completed successfully in 0 seconds.
29-05-2019 16:42:00 CST one INFO - output properties file=/export/servers/azkaban/azkaban-solo-server-0.1.0-SNAPSHOT/executions/3/one_output_3751669484555945824_tmp
29-05-2019 16:42:00 CST one INFO - Finishing job one at 1559119320455 with status SUCCEEDED
```

2. two-server 模式部署

2.1. 节点规划

HOST	角色
node-1	MySQL
Node-2	web-server 和 exec-server 不同进程

2.2. mysql 配置初始化

node-1:

```
mkdir /export/servers/azkaban
```

```
tar -zxvf azkaban-db-0.1.0-SNAPSHOT.tar.gz -C /export/servers/azkaban/
```

Mysql 上创建对应的库、增加权限、创建表

```
mysql> CREATE DATABASE azkaban_two_server; #创建数据库
```

```
mysql> use azkaban_two_server;
```

```
mysql> source /export/servers/azkaban/azkaban-db-0.1.0-SNAPSHOT/create-  
all-sql-0.1.0-SNAPSHOT.sql; #加载初始化 sql 创建表
```



2.3. web-server 服务器配置

node-2:

```
mkdir /export/servers/azkaban
```

```
tar -zxvf azkaban-web-server-0.1.0-SNAPSHOT.tar.gz -C /export/servers/azkaban/
```

```
tar -zxvf azkaban-exec-server-0.1.0-SNAPSHOT.tar.gz -C /export/servers/azkaban/
```

生成 ssl 证书:

```
keytool -keystore keystore -alias jetty -genkey -keyalg RSA
```

运行此命令后, 会提示输入当前生成 keystore 的密码及相应信息, 输入的密码请记住(所有密码统一以 123456 输入)。

完成上述工作后, 将在当前目录生成 keystore 证书文件, 将 keystore 拷贝到 azkaban web 服务器根目录中。

如: cp keystore azkaban-web-server-0.1.0-SNAPSHOT/

配置 conf/azkaban.properties:

```
# Azkaban Personalization Settings
azkaban.name=Test
azkaban.label=My Local Azkaban
azkaban.color=#FF3601
azkaban.default.servlet.path=/index
web.resource.dir=web/
default.timezone.id=Asia/Shanghai # 时区注意后面不要有空格

# Azkaban UserManager class
user.manager.class=azkaban.user.XmlUserManager
user.manager.xml.file=conf/azkaban-users.xml

# Azkaban Jetty server properties. 开启使用 ssl 并且知道端口
jetty.use.ssl=true
jetty.ssl.port=8443
jetty.maxThreads=25

# Azkaban Executor settings 指定本机 Executor 的运行端口
```



```
executor.host=localhost
executor.port=12321

# KeyStore for SSL ssl 相关配置 注意密码和证书路径
jetty.keystore=keystore
jetty.password=123456
jetty.keypassword=123456
jetty.truststore=keystore
jetty.trustpassword=123456

# Azkaban mysql settings by default. Users should configure their own username and password.
database.type=mysql
mysql.port=3306
mysql.host=node-1
mysql.database=azkaban_two_server
mysql.user=root
mysql.password=hadoop
mysql.numconnections=100

#Multiple Executor 设置为 false
azkaban.use.multiple.executors=true
#azkaban.executorselector.filters=StaticRemainingFlowSize,MinimumFreeMemory,CpuStatus
azkaban.executorselector.comparator.NumberOfAssignedFlowComparator=1
azkaban.executorselector.comparator.Memory=1
azkaban.executorselector.comparator.LastDispatched=1
azkaban.executorselector.comparator.CpuUsage=1
```

添加 azkaban.native.lib=false 和 execute.as.user=false 属性：

```
mkdir -p plugins/jobtypes
```

```
vim commonprivate.properties
```

```
azkaban.native.lib=false
```

```
execute.as.user=false
```

```
memCheck.enabled=false
```




2.4. exec-server 服务器配置

配置 conf/azkaban.properties:

```
# Azkaban Personalization Settings
azkaban.name=Test
azkaban.label=My Local Azkaban
azkaban.color=#FF3601
azkaban.default.servlet.path=/index
web.resource.dir=web/
default.timezone.id=Asia/Shanghai

# Azkaban UserManager class
user.manager.class=azkaban.user.XmlUserManager
user.manager.xml.file=conf/azkaban-users.xml

# Loader for projects
executor.global.properties=conf/global.properties
azkaban.project.dir=projects

# Where the Azkaban web server is located
azkaban.webserver.url=https://node-2:8443

# Azkaban mysql settings by default. Users should configure their own username and password.
database.type=mysql
mysql.port=3306
mysql.host=node-1
mysql.database=azkaban_two_server
mysql.user=root
mysql.password=hadoop
mysql.numconnections=100

# Azkaban Executor settings
executor.maxThreads=50
executor.port=12321
executor.flow.threads=30
```



2.5. 集群启动

先启动 exec-server

再启动 web-server。

启动 webServer 之后进程失败消失，可通过安装包根目录下对应启动日志进行排查。

```
while locating azkaban.webapp.AzkabanWebServer
Caused by: azkaban.executor.ExecutorManagerException: No active executor found
    at azkaban.executor.ExecutorManager.setupExecutors(ExecutorManager.java:253)
    at azkaban.executor.ExecutorManager.<init>(ExecutorManager.java:131)
    at azkaban.executor.ExecutorManager$$FastClassByGuice$$$1c1dfed.newInstance(<generated>)
    at com.google.inject.internal.DefaultConstructionProxyFactory$FastClassProxy.newInstance(
actory.java:89)
    at com.google.inject.internal.ConstructorInjector.provision(ConstructorInjector.java:111)
```

需要手动激活 executor

```
cd /export/servers/azkaban/execserver
```

```
curl -G "node-2:$(<./executor.port)/executor?action=activate" && echo
```

```
[root@node-2 execserver]# pwd
/export/servers/azkaban/execserver
[root@node-2 execserver]# curl -G "node-2:$(<./executor.port)/executor?action=activate" && echo
{"status": "success"}
[root@node-2 execserver]#
```

然后重新启动 webServer 就可以了。

3. multiple-executor 模式部署

multiple-executor 模式是多个 executor Server 分布在不同服务器上，只需要将 azkaban-exec-server 安装包拷贝到不同机器上即可组成分布式。

3.1. 节点规划

HOST	角色
Node-1	mysql
Node-2	web-server、exec-server
Node-3	exec-server

3.2. scp executor server 安装包到 node-3

启动之后，需要手动激活 executor

```
cd /export/servers/azkaban/execserver
```

```
curl -G "node-3:${(<./executor.port)}/executor?action=activate" && echo
```

Flow Execution 12 **SUCCEEDED**

Project test / Flow two / Execution 12

Graph Flow Trigger List Job List Flow Log Stats

Flow log

```
29-05-2019 19:53:36 CST two INFO - Assigned executor : node-2:12321
29-05-2019 19:53:36 CST two INFO - Running execid:12 flow:two project:1 version:1
29-05-2019 19:53:36 CST two INFO - Updating initial flow directory.
29-05-2019 19:53:36 CST two INFO - Fetching job and shared properties.
29-05-2019 19:53:36 CST two INFO - Starting flows
29-05-2019 19:53:36 CST two INFO - Running flow 'two'.
29-05-2019 19:53:36 CST two INFO - Configuring Azkaban metrics tracking for jobrunner object
29-05-2019 19:53:36 CST two INFO - Submitting job 'one' to run.
29-05-2019 19:53:36 CST two INFO - Created file appender for job one

29-05-2019 20:05:52 CST two INFO - Assigned executor : node-3:12321
29-05-2019 20:05:52 CST two INFO - Running execid:16 flow:two project:1 version:1
29-05-2019 20:05:52 CST two INFO - Updating initial flow directory.
29-05-2019 20:05:52 CST two INFO - Fetching job and shared properties.
29-05-2019 20:05:52 CST two INFO - Starting flows
29-05-2019 20:05:52 CST two INFO - Running flow 'two'.
29-05-2019 20:05:52 CST two INFO - Configuring Azkaban metrics tracking for jobrunner object
29-05-2019 20:05:52 CST two INFO - Submitting job 'one' to run.
29-05-2019 20:05:52 CST two INFO - Created file appender for job one
```

五、Azkaban 使用实战

1. shell command 调度

- 创建 job 描述文件

vi command.job

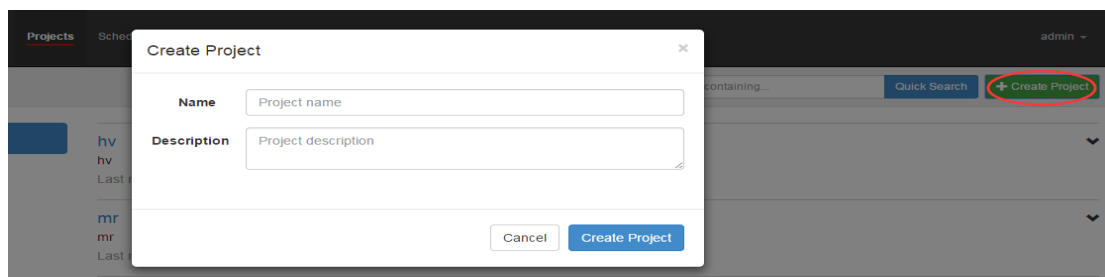
```
#command.job  
  
type=command  
  
command=echo 'hello'
```

- 将 job 资源文件打包成 zip 文件

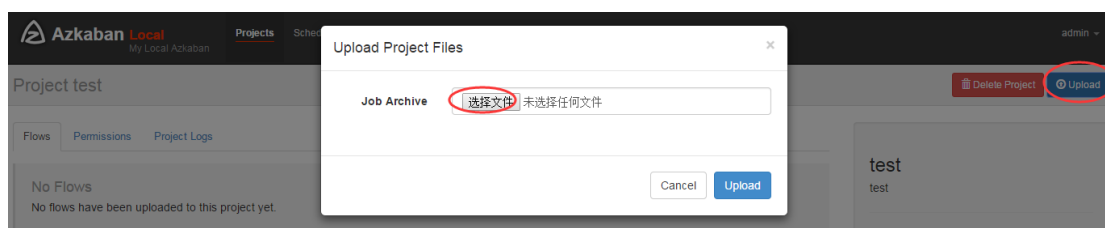
zip command.job

- 通过 azkaban 的 web 管理平台创建 project 并上传 job 压缩包

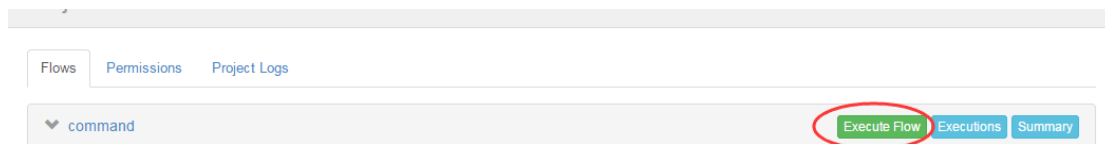
首先创建 Project



上传 zip 包



- 启动执行该 job



2. job 依赖调度

- 创建有依赖关系的多个 job 描述

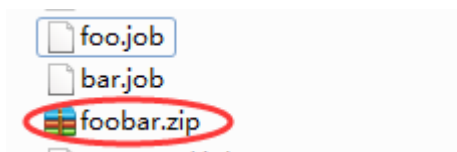
第一个 job: foo.job

```
# foo.job  
  
type=command  
  
command=echo foo
```

第二个 job: bar.job 依赖 foo.job

```
# bar.job  
  
type=command  
  
dependencies=foo  
  
command=echo bar
```

- 将所有 job 资源文件打到一个 zip 包中



- 在 azkaban 的 web 管理界面创建工程并上传 zip 包
- 启动工作流 flow

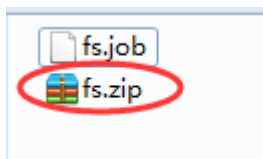


3. HDFS 任务调度

- 创建 job 描述文件

```
# fs.job  
  
type=command  
  
command=/home/hadoop/apps/hadoop-2.6.1/bin/hadoop fs -mkdir /azaz
```

- 将 job 资源文件打包成 zip 文件



- 通过 azkaban 的 web 管理平台创建 project 并上传 job 压缩包
- 启动执行该 job

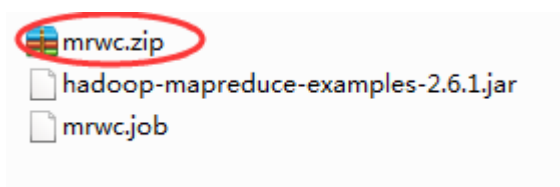
4. MAPREDUCE 任务调度

mr 任务依然可以使用 command 的 job 类型来执行

- 创建 job 描述文件，及 mr 程序 jar 包（示例中直接使用 hadoop 自带的 example jar）

```
# mrwc.job  
  
type=command  
  
command=/home/hadoop/apps/hadoop-2.6.1/bin/hadoop jar hadoop-mapreduce-  
examples-2.6.1.jar wordcount /wordcount/input /wordcount/azout
```

- 将所有 job 资源文件打到一个 zip 包中



- 在 azkaban 的 web 管理界面创建工程并上传 zip 包
- 启动 job



5. HIVE 脚本任务调度

- 创建 job 描述文件和 hive 脚本

Hive 脚本: test.sql

```
use default;

drop table aztest;

create table aztest(id int,name string) row format delimited fields terminated
by ',';

load data inpath '/aztest/hiveinput' into table aztest;

create table azres as select * from aztest;

insert overwrite directory '/aztest/hiveoutput' select count(1) from aztest;
```

Job 描述文件: hivef.job

```
# hivef.job

type=command

command=/home/hadoop/apps/hive/bin/hive -f 'test.sql'
```

- 将所有 job 资源文件打到一个 zip 包中创建工程并上传 zip 包, 启动 job

6. 定时任务调度

除了手动立即执行 workflow 任务外，azkaban 也支持配置定时任务调度。开启方式如下：

首页选择待处理的 project

The screenshot shows the Azkaban web interface. At the top, the 'Projects' tab is selected and highlighted with a red box. Below it, a search bar and a 'Quick Search' button are visible. A red arrow points from the 'Projects' tab to a project entry labeled 'demo', which is also highlighted with a red box. Below the project name, it says 'itcast' and 'Last modified on 2019-06-06 16:50:08 by azkaban.' Below this, the 'Project demo' section is shown with tabs for 'Flows', 'Permissions', and 'Project Logs'. The 'Flows' tab is active, and a flow named 'two' is highlighted with a red box. To the right of the flow name are three buttons: 'Execute Flow', 'Executions', and 'Summary', with the 'Summary' button highlighted by a red box. A red arrow points from the 'two' flow to the 'Summary' button, with the text '两个任选其一 点击进入' (Choose either one, click to enter) below it. Below the flow list, a 'Schedule / Execute Flow' button is highlighted with a red box. A red arrow points from this button to a workflow diagram. The diagram shows a 'READY (command)' node pointing to a box labeled 'one', which points to a box labeled 'two'.



上述图片中，选择左边 **schedule** 表示配置定时调度信息，选择右边 **execute** 表示立即执行工作流任务。



Schedule Flow Options

首先要确定时区是否正确

All schedules are based on the server timezone: **Asia/Shanghai.**

Warning: the execution will be skipped if it is scheduled to run during the hour that is lost when DST starts in the Spring. E.g. there is no 2 - 3 AM when PST switches to PDT.

Min 分 *

Hours 时 *

Day of Month 日 ?

Month 月 *

Day of Week 周 *

Year 年

Special Characters:

* any value

, value list separators

- range of values

/ step values

[Detailed instructions.](#)

填写规则说明

最终表达式显示区域

0 * * ? * *

Reset

Next 10 scheduled executions for this cron expression only:

- 2019-06-08T12:09:00
- 2019-06-08T12:10:00
- 2019-06-08T12:11:00
- 2019-06-08T12:12:00
- 2019-06-08T12:13:00
- 2019-06-08T12:14:00
- 2019-06-08T12:15:00
- 2019-06-08T12:16:00
- 2019-06-08T12:17:00
- 2019-06-08T12:18:00

显示接下来的定时执行时间
可用于验证配置是否正确